

DSA STUDY BLUEPRINT SERIES

93. Validate Binary Search Tree

Validating Binary Search Tree constraints

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Validation:** Validate BST rules recursively.
- Pass down minimum and maximum boundary values to child nodes.

Memory & Execution Blueprint

For node 3: check if `minVal < 3 < maxVal` recursively.

C++ Implementation Reference

Standard code layout and syntax

```
bool validate(TreeNode* root, long minVal, long maxVal) {  
    if (!root) return true;  
    if (root->val <= minVal || root->val >= maxVal) return false;  
    return validate(root->left, minVal, root->val) && validate(root->right, root->val, maxVal);  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(H)$

Key Practices & Gotchas

- **Important:** Use long limits (`LONG_MIN` , `LONG_MAX`) to handle integer overflow conditions.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.